

SECURE SOCKET PROGRAMMING

Byeong-hee Roh

Dept. of Software and Computer Engineering

Ajou University



변화와 융합의 중심
MMcN

멀티미디어 융합 연구실

Mobile Multimedia convergene Network Lab.
<http://mmcn.ajou.ac.kr>



Contents

- ❖ Transport Layer Security Protocols
- ❖ OpenSSL
- ❖ Example: Echo SSL Client-Server

TRANSPORT LAYER SECURITY PROTOCOLS



HTTPS – Security Concerns on HTTP

❖ Privacy

- HTTP messages are written in ordinary ASCII text,
- So anyone can see content of HTTP messages

❖ Integrity

- no encryption for HTTP messages is used
- HTTP is insecure
 - It is subject to man in the middle attack and
- eavesdropping of sensitive information.

❖ Authentication

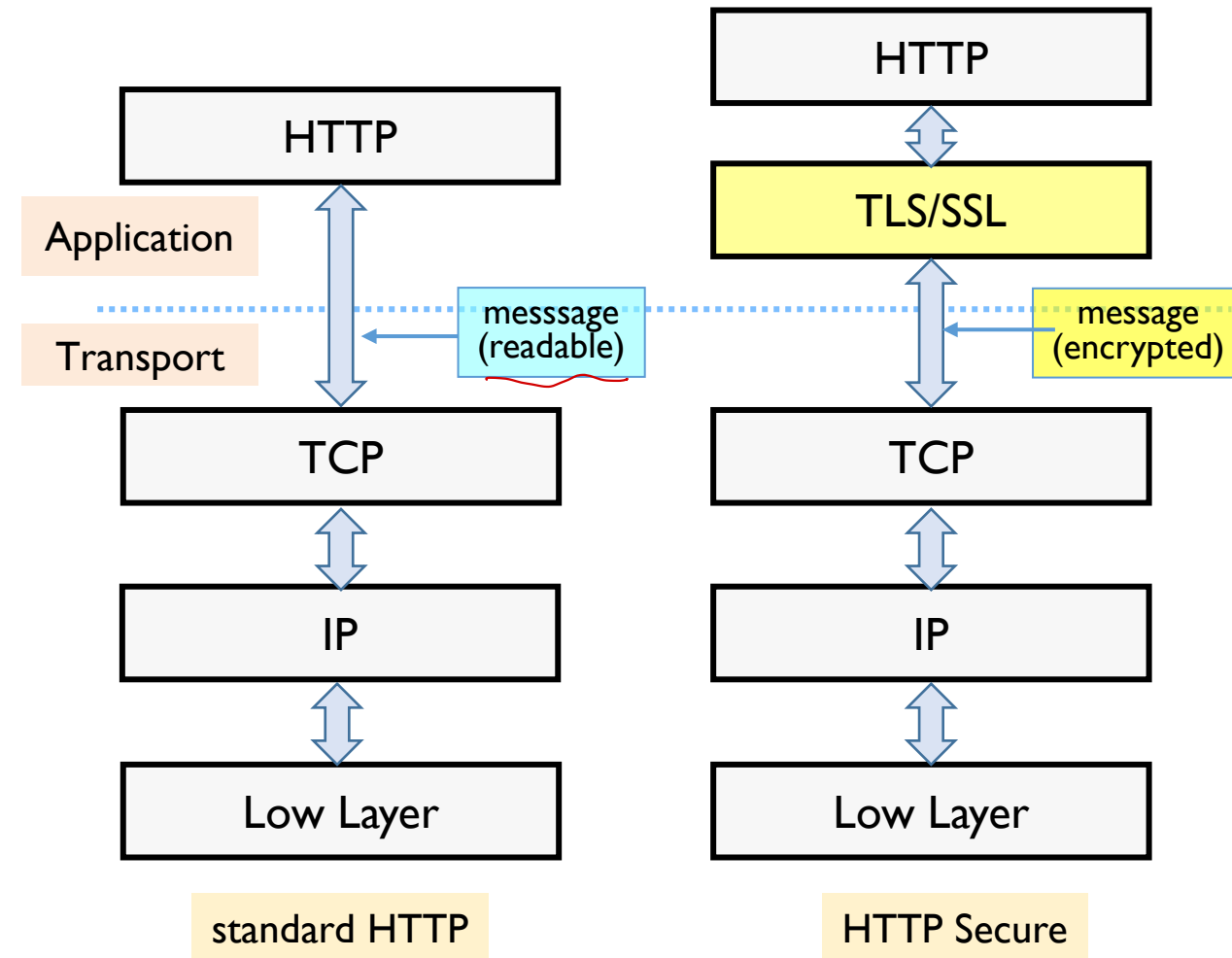
- Not clear who you are talking with
 - Messages for authentication with the username and password can be intercepted by malicious users

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Firefox/3.6.10\r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Accept-Charset: ISO-8859-1,utf-8;q=0.7\r\n
Keep-Alive: 115\r\n
Connection: keep-alive\r\n
\r\n
```

HTTPS – HTTP Secure

❖ HTTP Secure

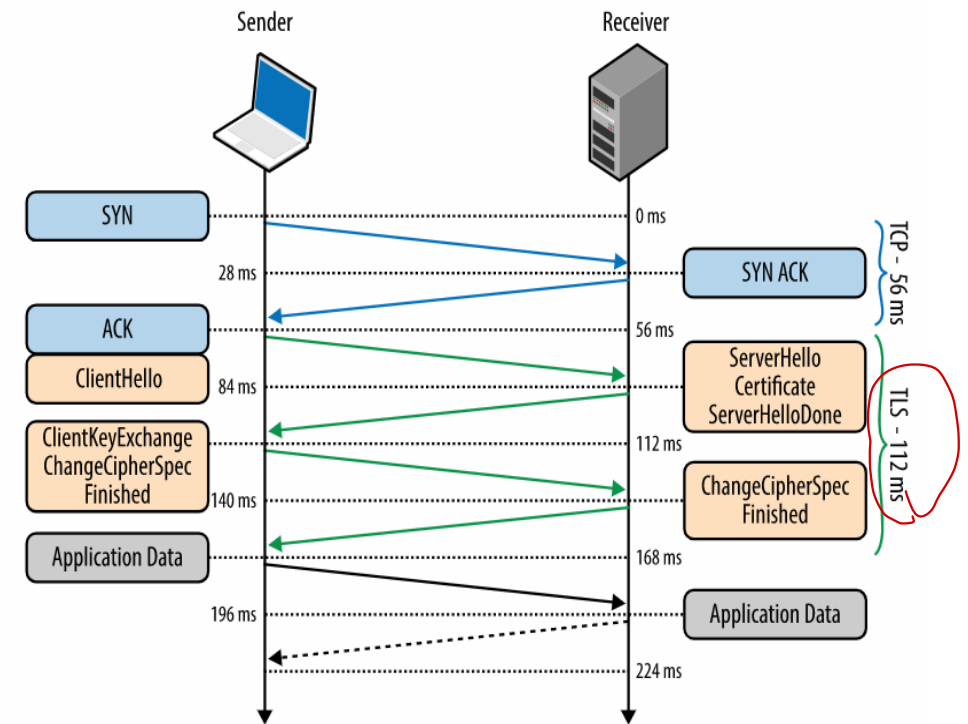
- an extension of the HTTP for secure communication over a computer network
- sublayer between HTTP and TCP is embedded
 - SSL (Secure Socket Layer), formerly
 - TLS (Transport Layer Security)



HTTPS – Handshake

❖ HTTP Secure

- TLS Handshake
 - before exchanging data over TLS
 - the encrypted tunnel is configured,
 - then encrypted data are exchanged
- 3RTTs before HTTP message exchange
 - TCP connection – 1 RTT
 - TLS 1.2 connection – 2 RTT
 - HTTP requests/response over TCP



Source: <https://hpbn.co/transport-layer-security-tls/>

Transport Layer Security Protocols

❖ SSL (Secure Socket Layer)

- SSL v1.0 (1994) → deprecated
 - developed by Netscape Communications, but it was never released due to security flaws.
- SSL v2.0 (1995) → deprecated
 - the first publicly released version, but it also had significant vulnerabilities.
- SSL v3.0 (1996) → deprecated
 - developed a new encryption algorithm RC-40 to fix the safety vulnerabilities of SSL v2.0
 - widely adopted SSL version
- Shutdown (2020)
 - SSL 2.0 and SSL 3.0 are formally deprecated due to known vulnerabilities
 - TLS fully replaced SSL in most implementations

Transport Layer Security Protocols

❖ TLS (Transport Layer Security)

- TLS 1.0 (1999) → deprecated
 - released by IETF as an improvement over SSL 3.0
 - fixed some cryptographic flaws while maintaining backward compatibility
- TLS 1.1 (2006) → deprecated
 - enhancing security with protection against cipher block chaining (CBC) attacks
- TLS 1.2 (2008) → in use
 - released with stronger cryptographic algorithms, improved security, and flexibility in cipher suites
- TLS 1.3 (2018) → in use (openSSL)
 - simplifying the protocol by removing obsolete and vulnerable features, providing faster performance and enhanced security
- Present:
 - TLS 1.2 and TLS 1.3 are widely used, with ongoing efforts to improve and secure internet communications.

Transport Layer Security Protocols

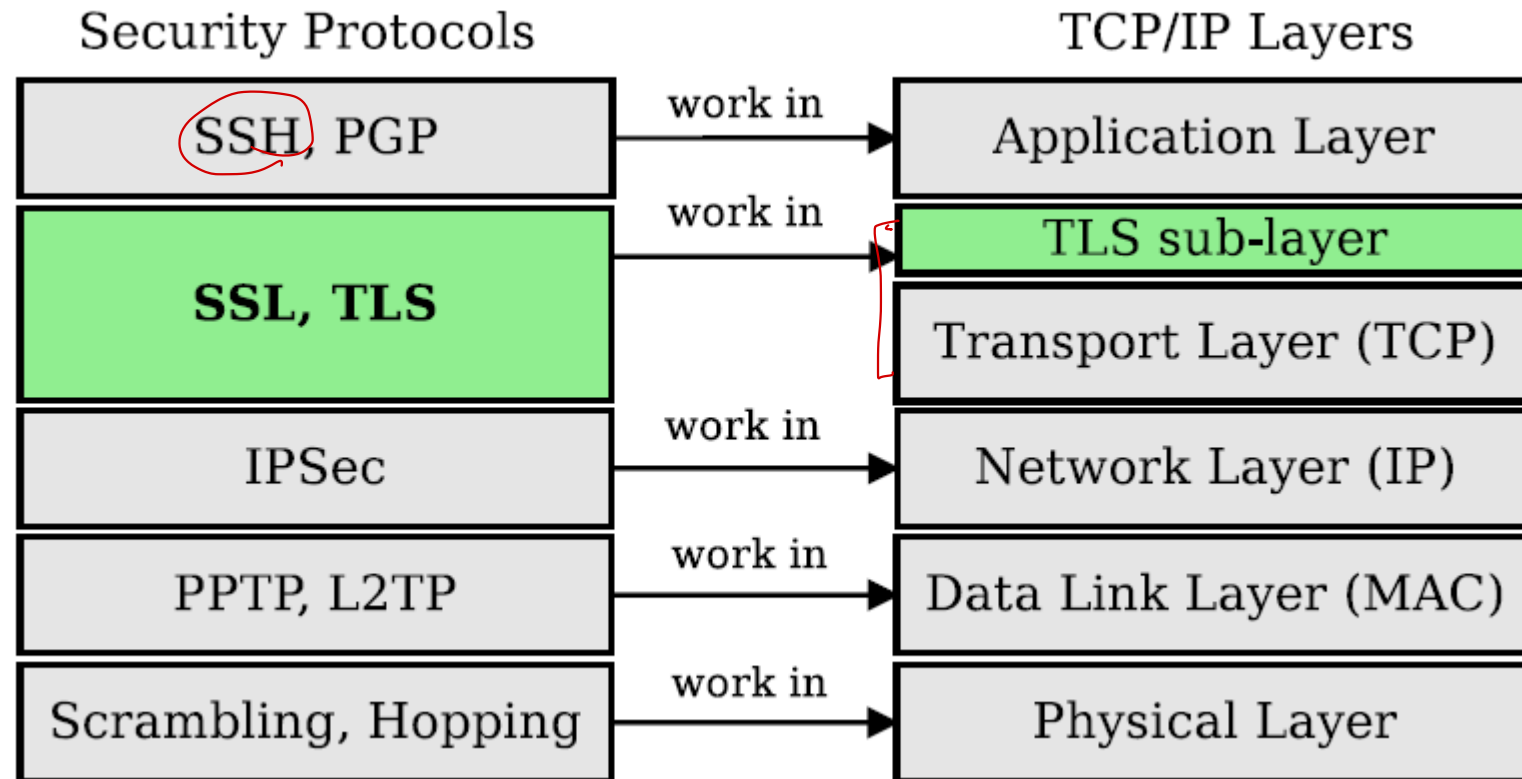
❖ Comparison: SSL vs. TLS

Attribute	SSL (Secure Socket Layer)	TLS (Transport Layer Security)
Origin Year	1995	1999
Developed By	Netscape	Internet Engineering Task Force (IETF)
Current Version	SSL 3.0 (1996) – Last and most used version	TLS 1.3 (2018)
Key Length	Typically up to 128-bit	Up to 256-bit (stronger encryption)
Cryptographic Algorithms	Relied more on older algorithms	Uses modern algorithms, phasing out deprecated ones
Vulnerabilities	Susceptible to issues like POODLE attacks	Designed to address past vulnerabilities of SSL
Handshake Process	Slower due to fewer optimizations	Streamlined and faster
Support	Phased out by many modern browsers and platforms	Widely supported and recommended
Interoperability	Limited with modern systems	High with current systems and platforms
Use Case	Historical; foundational for secure connections	Present and future; standard for secure connections

Transport Layer Security Protocols

❖ Security-Related Protocols with TCP/IP architecture

- SSH (Secure Shell)
- PGP (Pretty Good Privacy)
- IPsec (IP Security)
- PPTP (Point-to-Point Tunneling Protocol)
- L2TP (Layer 2 Tunneling Protocol)



Transport Layer Security Protocols

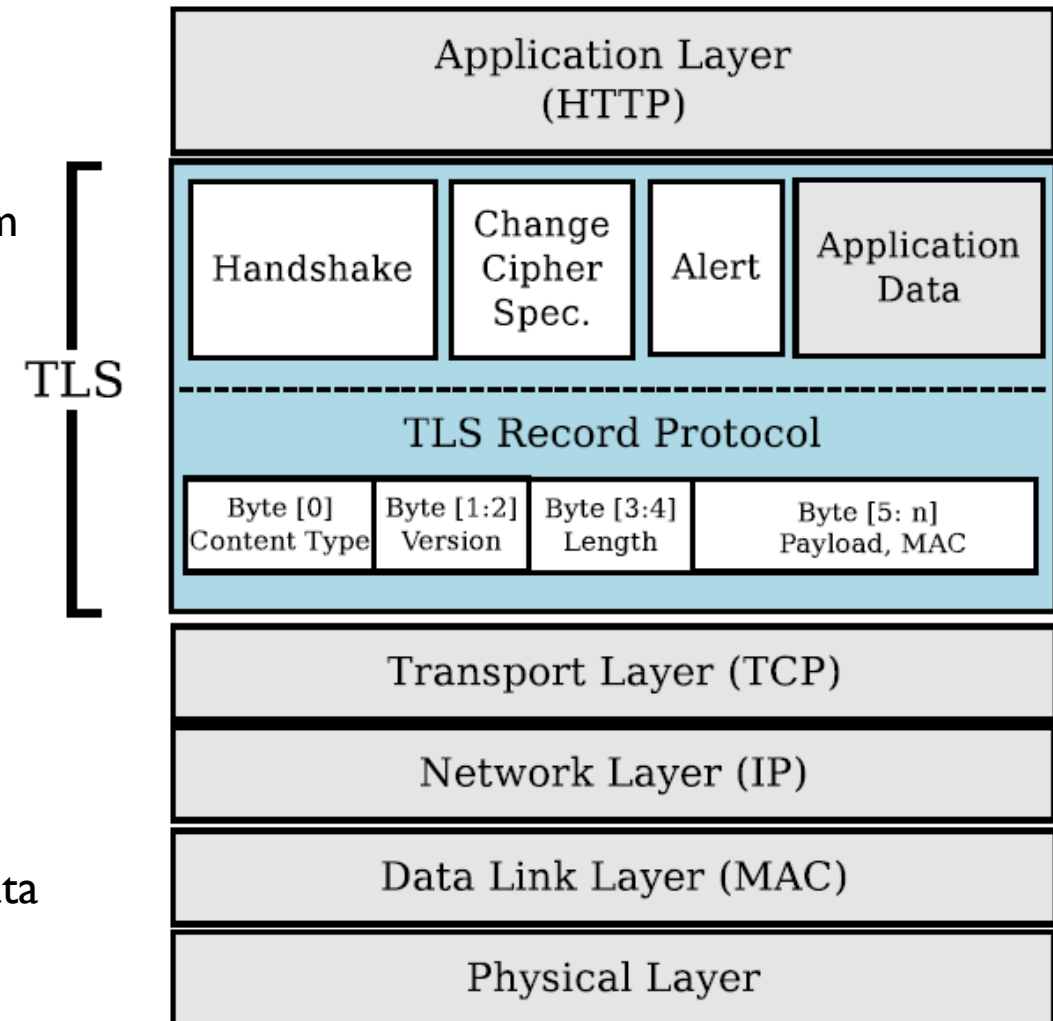
❖ TLS Layers and Sub-protocols

■ Top Layer

- used to manage TLS exchanges
 - to allow peers to agree on an encryption algorithm and a shared secret key
 - to authenticate themselves, and
 - to report errors to each other.
- three handshaking sub-protocols
 - Handshake
 - Change Cipher Specification
 - Alert

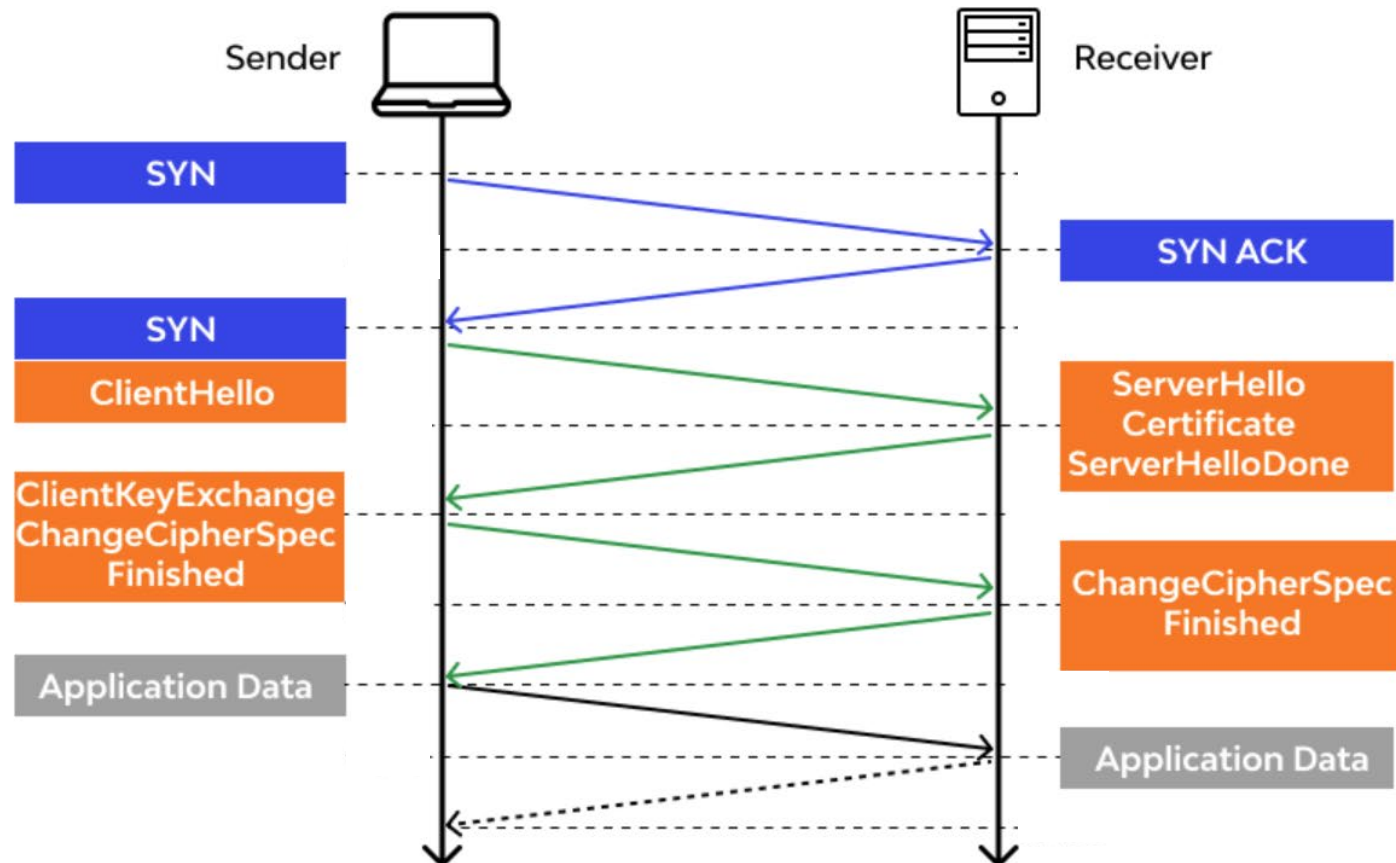
■ Low Layer

- TLS Record protocol
 - can be presented as an envelope for application data and TLS messages from the top layer protocols
 - split data into chunks, authenticated with MAC, encrypted and finally transmitted



Transport Layer Security Protocols

❖ TLS Handshake Sequence

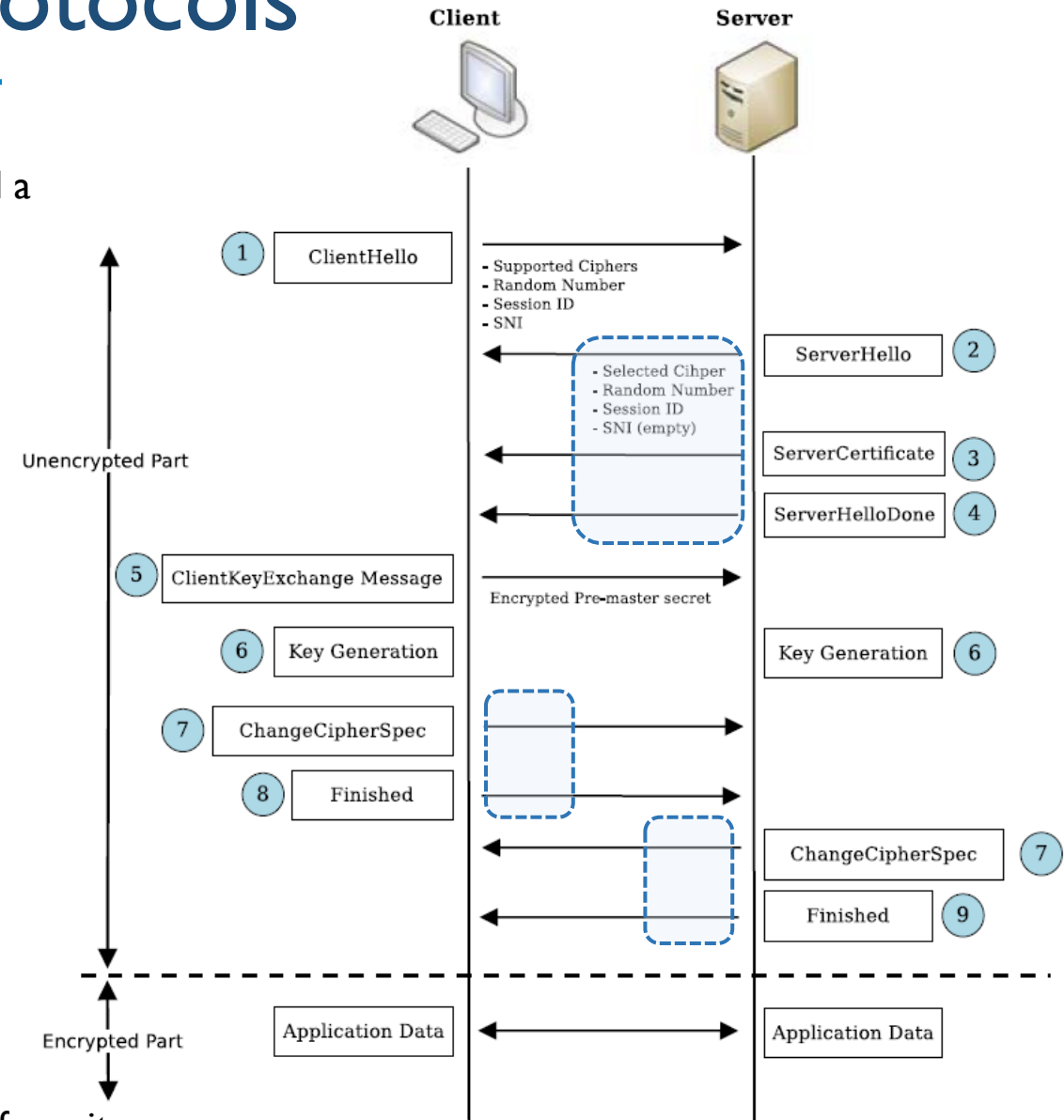


Transport Layer Security Protocols

1. **ClientHello** : contains the usable cipher suites, supported extensions and a random number
2. **ServerHello** : holds the selected cipher, supported extensions and a random number.
3. **ServerCertificate** : contains a certificate with a server public key.
4. **ServerHelloDone** : indicates the end of the ServerHello and associated messages. If the client receives a request for its certificate, it sends a ClientCertificate message.
5. **ClientKeyExchange** : Based on the server random number, the client generates a random Pre-Master Secret, encrypts it with the public key given in the server's certificate and sends it to the server.
6. **Key Generation** : Both client and server generate a master secret from the Pre-Master Secret and exchanged random values.
7. **ChangeCipherSpec** : is exchanged between the client and the server to start using the new keys for encryption.
8. **Finished** : The client sends the Finished message to verify that the key exchange and authentication processes were successful.
9. **Finished** : The server sends the Finished message to the client to end the handshake phase.



Once both sides have received and validated the Finished message from its peer, they can exchange encrypted application data over **the new TLS connection**.



Transport Layer Security Protocols

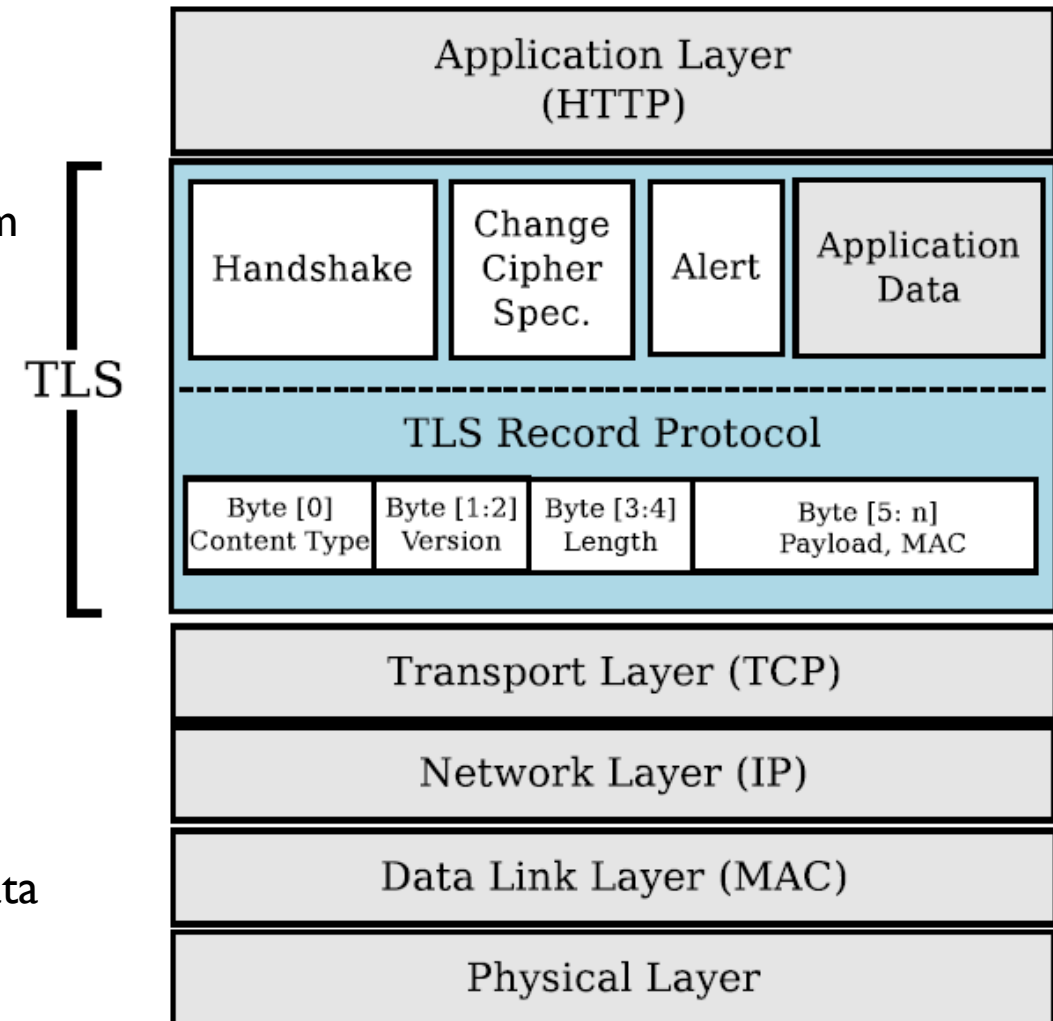
❖ TLS Layers and Sub-protocols

■ Top Layer

- used to manage TLS exchanges
 - to allow peers to agree on an encryption algorithm and a shared secret key
 - to authenticate themselves, and
 - to report errors to each other.
- three handshaking sub-protocols
 - Handshake
 - Change Cipher Specification
 - Alert

■ Low Layer

- TLS Record protocol
 - can be presented as an envelope for application data and TLS messages from the top layer protocols
 - split data into chunks, authenticated with MAC, encrypted and finally transmitted





OPENSSL



OpenSSL

❖ OpenSSL

- is an open-source software library for SSL/TLS-based secure network comm.
- provides robust tools for TLS/SSL and general-purpose cryptography
- includes the OpenSSL Toolkit used to
 - support SSL/TLS handshake procedure,
 - generate private keys,
 - create CSRs (Certificate Signing Requests), and
 - handle digital certificates
- History
 - SSLeay (1998): officially launched as a fork of SSLeay, an earlier SSL implementation.
 - OpenSSL 0.9.6 (2000) supports for SSL 3.0 and TLS 1.0
 - OpenSSL 0.9.7 (2003) supports for TLS 1.1
 - OpenSSL 1.0.0 (2010) supports for TLS 1.2
 - OpenSSL 1.1.1 (2018) supports for full TLS 1.3
 - OpenSSL 3.1.x (2023) improves TLS 1.3 and enhances security more



OpenSSL

❖ Getting OpenSSL

■ for Linux or MacOS

- Installed by default during OS installation
- version check

```
~$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
~$
```

■ for windows

- not installed by default
- can be downloaded from Shining Light Productions
 - <https://slproweb.com/products/Win32OpenSSL.html>
 - OpenSSL 3.3.2 is available (on Sep., 2024)

OpenSSL

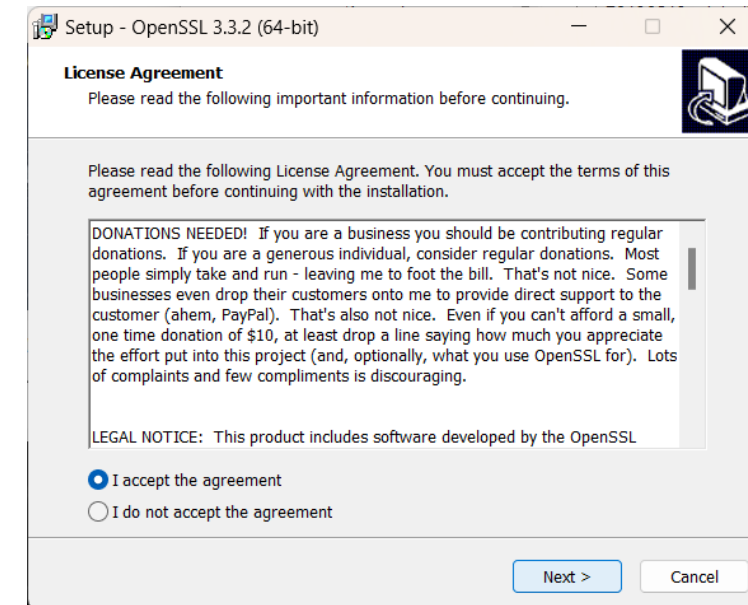
❖ Install OpenSSL on Windows OS (1/3)

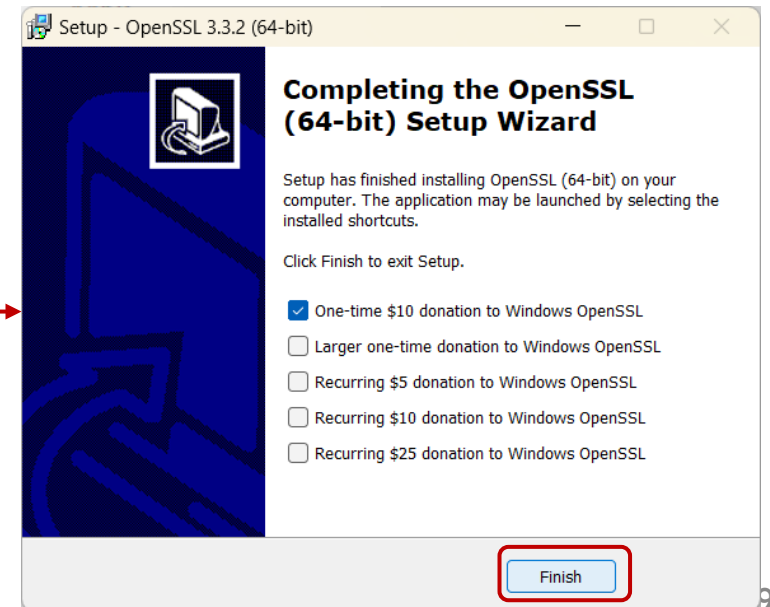
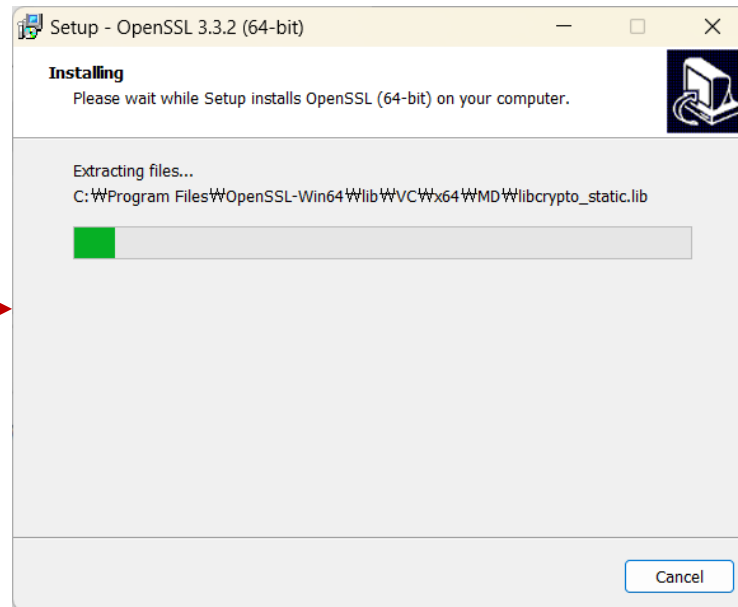
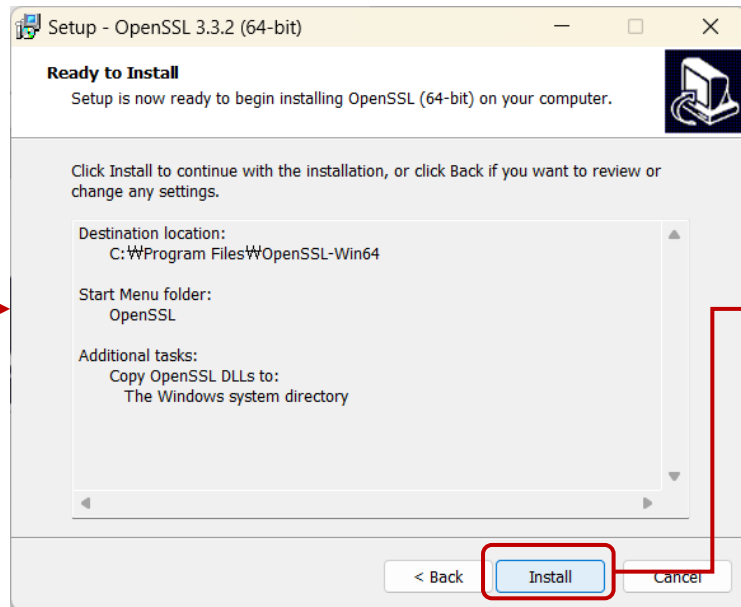
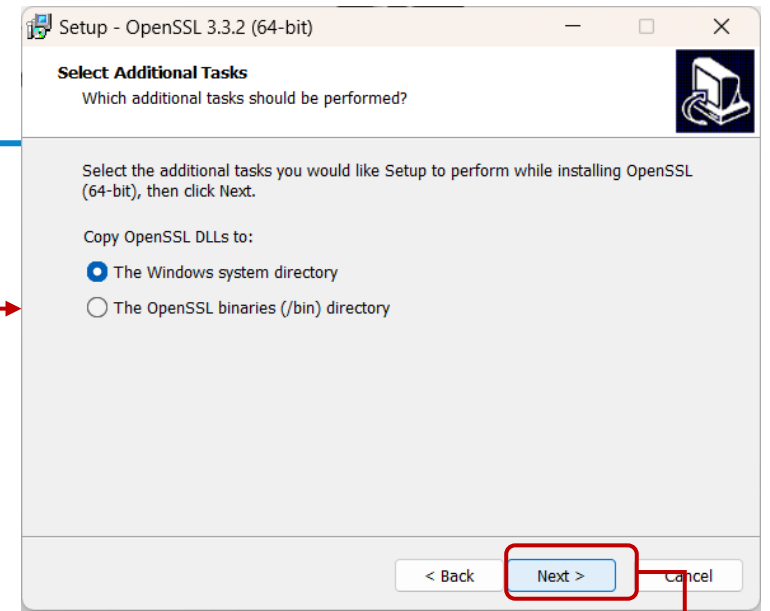
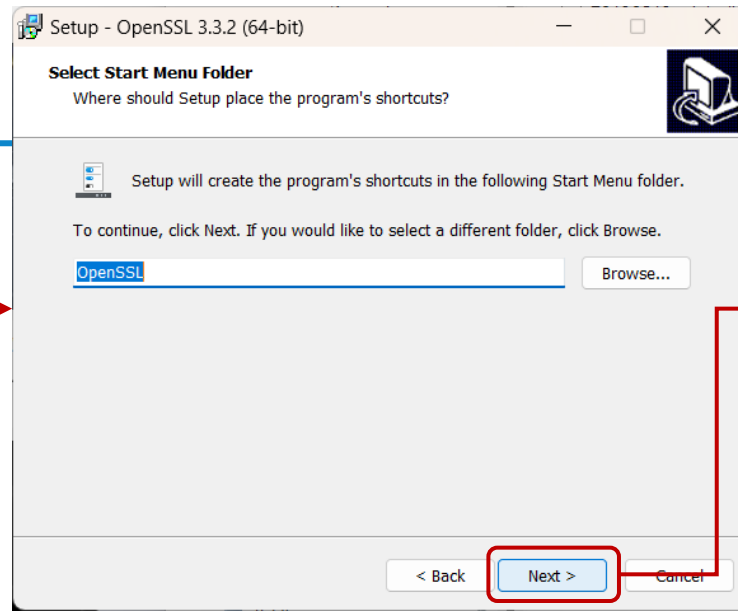
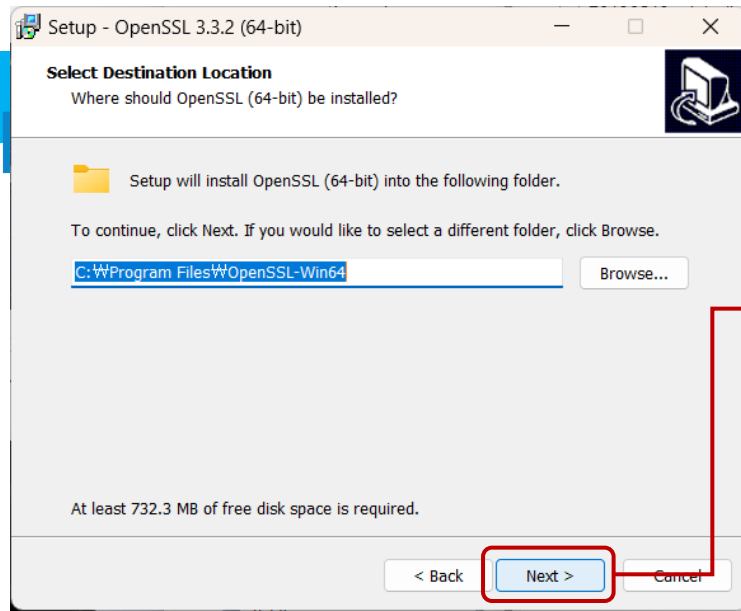
- download installation file, and run

Download Win32/Win64 OpenSSL

Download Win32/Win64 OpenSSL today using the links below!

File	Type	Description
Win64 OpenSSL v3.3.2 Light EXE MSI	5MB Installer	Installs the most commonly used essentials of Win64 OpenSSL v3.3.2 (Recommended for users by the creators of OpenSSL). Only installs on 64-bit versions of Windows and targets Intel x64 chipsets. Note that this is a default build of OpenSSL and is subject to local and state laws. More information can be found in the legal agreement of the installation.
Win64 OpenSSL v3.3.2 EXE MSI	217MB Installer	Installs Win64 OpenSSL v3.3.2 (Recommended for software developers by the creators of OpenSSL). Only installs on 64-bit versions of Windows and targets Intel x64 chipsets. Note that this is a default build of OpenSSL and is subject to local and state laws. More information can be found in the legal agreement of the installation.
Win32 OpenSSL v3.3.2 Light EXE MSI	4MB Installer	Installs the most commonly used essentials of Win32 OpenSSL v3.3.2 (Only install this if you need 32-bit OpenSSL for Windows). Note that this is a default build of OpenSSL and is subject to local and state laws. More information can be found in the legal agreement of the installation.
Win32 OpenSSL v3.3.2 EXE MSI	175MB Installer	Installs Win32 OpenSSL v3.3.2 (Only install this if you need 32-bit OpenSSL for Windows). Note that this is a default build of OpenSSL and is subject to local and state laws. More information can be found in the legal agreement of the installation.
Win64 OpenSSL v3.3.2 Light for ARM (EXPERIMENTAL) EXE MSI	6MB Installer	Installs the most commonly used essentials of Win64 OpenSSL v3.3.2 for ARM64 devices (Only install this VERY EXPERIMENTAL build if you want to try 64-bit OpenSSL for Windows on ARM processors). Note that this is a default build of OpenSSL and is subject to local and state laws. More information can be found in the legal agreement of the installation.
Win64 OpenSSL v3.3.2 for ARM (EXPERIMENTAL) EXE MSI	170MB Installer	Installs Win64 OpenSSL v3.3.2 for ARM64 devices (Only install this VERY EXPERIMENTAL build if you want to try 64-bit OpenSSL for Windows on ARM processors). Note that this is a default build of OpenSSL and is subject to local and state laws. More information can be found in the legal agreement of the installation.



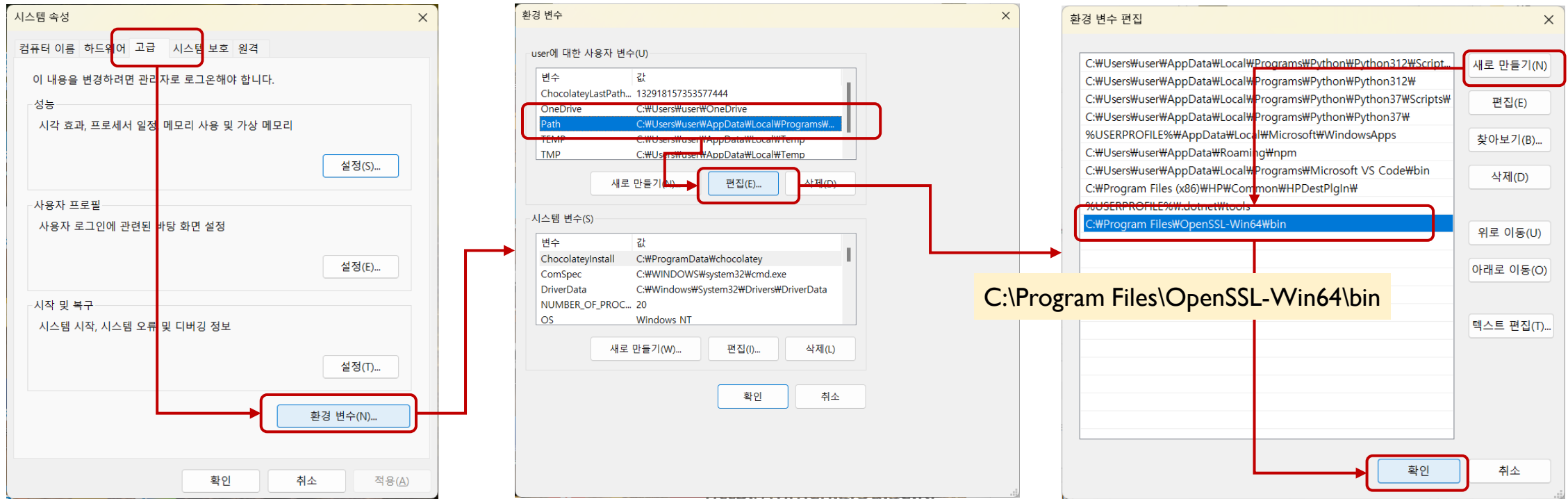


OpenSSL

❖ Install OpenSSL on Windows OS (3/3)

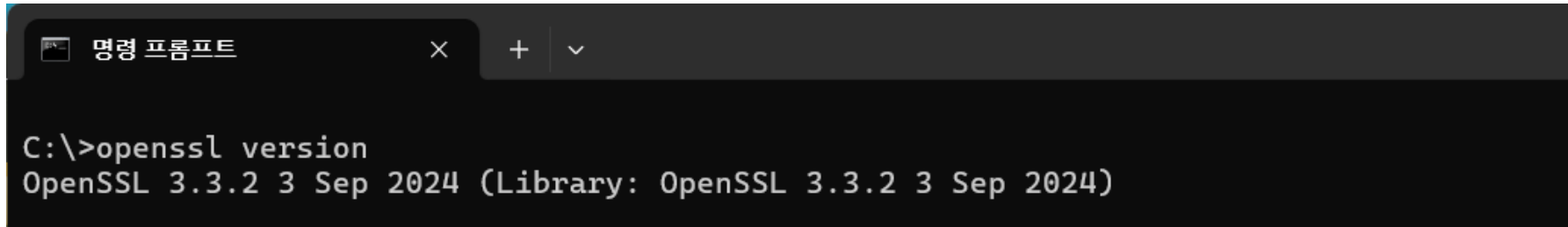
- configure the Path variable in Windows
 - open System Properties Window

```
C:\Windows\System32>sysdm.cpl
```



OpenSSL

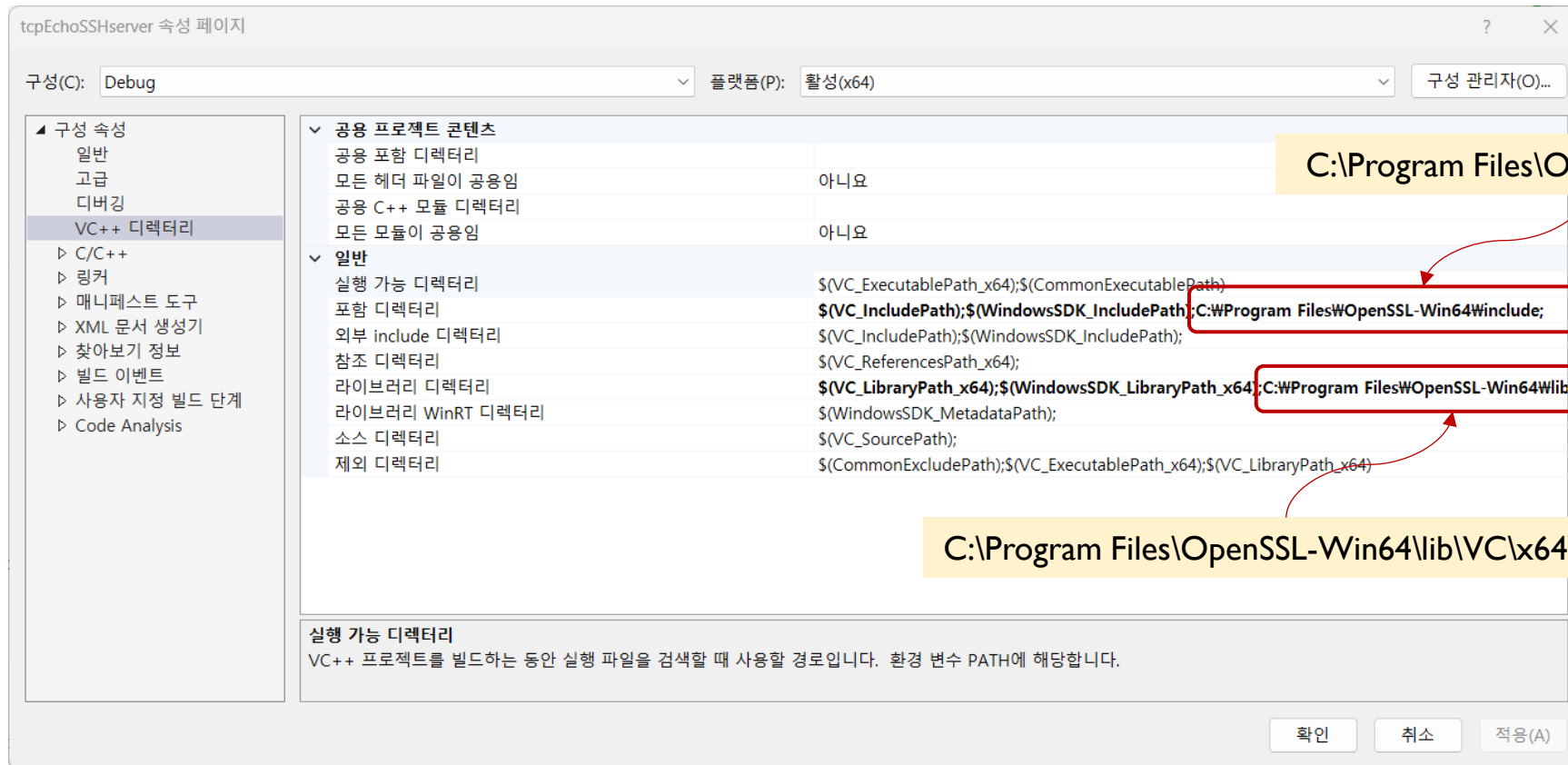
❖ check OpenSSL version



```
명령 프롬프트
C:\>openssl version
OpenSSL 3.3.2 3 Sep 2024 (Library: OpenSSL 3.3.2 3 Sep 2024)
```

OpenSSL

❖ Configure Visual Studio 2020 Compiling Environments ▪ in Project Properties Window

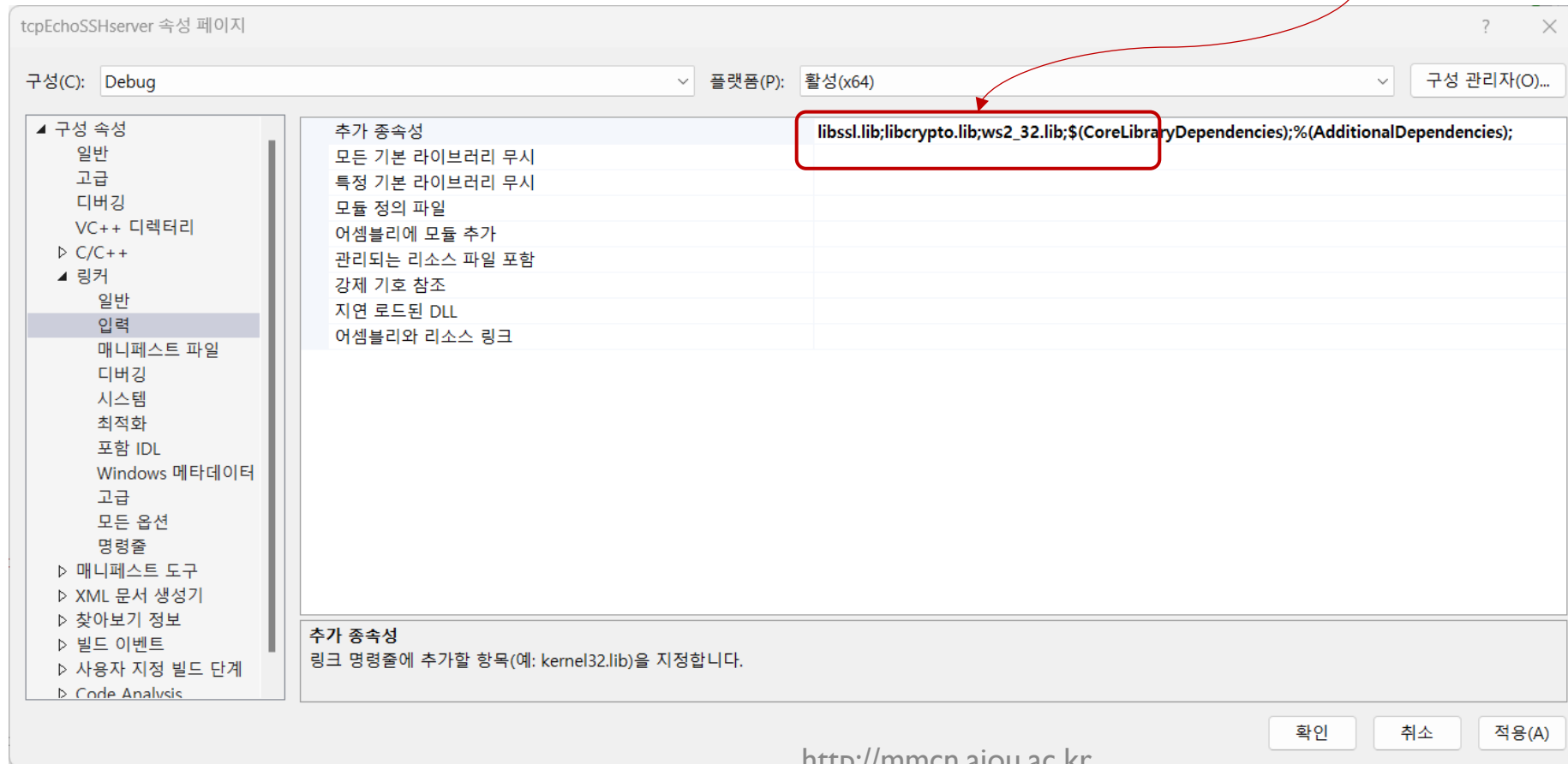


OpenSSL

❖ Configure Visual Studio 2020 Compiling Environments

■ in Project Properties Window

libssl.lib; libcrypto.lib; ws2_32.lib;





OpenSSL Programming

- ❖ Basic Steps for OpenSSL Programming
 1. Initialize the OpenSSL Library
 2. Create an SSL Context
 3. Set Certificate and Private Key (for server)
 4. Create an SSL Connection
 5. Send and Receive Data over SSL
 6. Shutdown the SSL Connection and Cleanup



OpenSSL Programming

❖ Step 1: Initializing OpenSSL

- Before using OpenSSL, the library must be initialized using functions
 - `SSL_load_error_strings()`
 - registers the error strings for all `libcrypto` functions, and registers the `libssl` error strings
 - `OpenSSL_add_ssl_algorithms()`
 - initialize SSL library by registering algorithms

```
1 #include <openssl/ssl.h>
2 #include <openssl/err.h>
3
4 void InitOpenSSL() {
5     SSL_load_error_strings();    // Load error strings for human-readable error messages
6     OpenSSL_add_ssl_algorithms(); // Initialize OpenSSL algorithms
7 }
```

OpenSSL Programming

❖ Step 2: Create an SSL Context

- it creates an SSL context to manage SSL connections.
- for servers
 - `SSLv23_server_method()` is used

```
1 SSL_CTX* CreateServerContext() {  
2     const SSL_METHOD *method = SSLv23_server_method(); // SSL method for servers  
3     SSL_CTX *ctx = SSL_CTX_new(method);                // Create SSL context  
4  
5     return ctx;  
6 }
```

- for clients
 - `SSLv23_client_method()` is used to create the context.

```
1 SSL_CTX* CreateClientContext() {  
2     const SSL_METHOD *method = SSLv23_client_method(); // SSL method for clients  
3     SSL_CTX *ctx = SSL_CTX_new(method);                // Create SSL context  
4  
5     return ctx;  
6 }
```

OpenSSL Programming

❖ Step 3: Set Certificate and Private Key (for server) (1/2)

- Server sets up a certificates and private keys into the SSL_CTX or SSL object, respectively for SSL comm. with clients.
- using functions
 - `SSL_CTX_use_certificate_file()`
 - `SSL_CTX_use_PrivateKey_file()`

```
1 void ConfigureServerContext(SSL_CTX *ctx) {
2     // Set the server certificate
3     if (SSL_CTX_use_certificate_file(ctx, "server.crt", SSL_FILETYPE_PEM) <= 0) {
4         ERR_print_errors_fp(stderr);
5         exit(EXIT_FAILURE);
6     }
7
8     // Set the server private key
9     if (SSL_CTX_use_PrivateKey_file(ctx, "server.key", SSL_FILETYPE_PEM) <= 0) {
10        ERR_print_errors_fp(stderr);
11        exit(EXIT_FAILURE);
12    }
13 }
```

OpenSSL Programming

❖ Step 3: Set Certificate and Private Key (for server) (2/2)

- to create the certificates and private keys at server

```
openssl req -x509 -newkey rsa:4096 -keyout server.key -out server.crt -days 365
```

- it needs a password for the keys
- server.key and server.crt are created
 - the files must be in the same folder as the executable file.

- to create the certificates and private keys without password

```
openssl req -x509 -newkey rsa:4096 -node -keyout server.key -out server.crt -days 365
```

- the following fields are requested, but ignore, i.e. just <Enter>

```
Country Name (2 letter code) [AU]:          <--- just input <Enter>
State or Province Name (full name) [Some-State]:
Locality Name (eg, city) []:
Organization Name (eg, company) [Internet Widgits Pty Ltd]:
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:
Email Address []:
```

OpenSSL Programming

❖ Step 4: Create an SSL Connection

- After the server accepts a connection
 - the socket file descriptor is linked to the SSL structure.
- SSL handshake process is done using
 - `SSL_accept()` at server
 - `SSL_connect()` at client
- example
 - at server

```
1 int server_fd;           // Socket file descriptor
2 SSL *ssl = SSL_new(ctx); // Create new SSL structure
3 SSL_set_fd(ssl, server_fd); // Associate SSL with socket descriptor
```

- at client

```
1 int client_fd;           // Socket file descriptor
2 SSL *ssl = SSL_new(ctx); // Create new SSL structure
3 SSL_set_fd(ssl, client_fd); // Associate SSL with socket descriptor
```

OpenSSL Programming

❖ Step 5: Send and Receive Data over SSL connection (1/2)

- the following functions are used to send and receive data over SSL conn.
 - `SSL_write()` for sending, and
 - `SSL_read()` for receiving data
 - cf. `send()` and `recv()` with normal sockets.
- Syntax

```
#include <openssl/ssl.h>
int SSL_write(SSL *ssl, const void *buf, int num);
int SSL_read(SSL *ssl, void *buf, int num);
```

- `ssl` : SSL descriptor
- `buf` : the buffer pointer holding the data
- `num` : the length of the data (in bytes)

OpenSSL Programming

❖ Step 5: Send and Receive Data over SSL connection (2/2)

▪ example

- to send a data to the established SSL connection

```
1  const char *msg = "Hello, SSL!";  
2  
3  SSL_write(ssl, msg, strlen(msg)); // Send data over SSL connection
```

- to read a data from the established SSL connection

```
1  char buffer[1024];  
2  
3  int bytes = SSL_read(ssl, buffer, sizeof(buffer)); // Receive data over SSL connection  
4  
5  buffer[bytes] = '\0'; // Null-terminate the received data  
6  printf("Received: %s\n", buffer);
```

OpenSSL Programming

❖ Step 6: Shutdown the SSL Connection and Cleanup

- When communication is finished, the SSL connection should be shut down and resources cleaned up using
 - `SSL_shutdown()` closes the SSL connection,
 - `SSL_free()` releases the SSL structure, and
 - `SSL_CTX_free()` releases the context to avoid memory leaks
- Syntax

```
SSL_CTX* ctx;
```

```
SSL* ssl;
```

```
SSL_shutdown(ssl); // Shutdown the SSL connection
```

```
SSL_free(ssl);      // Free the SSL structure
```

```
SSL_CTX_free(ctx);  // Free the SSL context
```


EXAMPLE: ECHO SSL CLIENT- SERVER

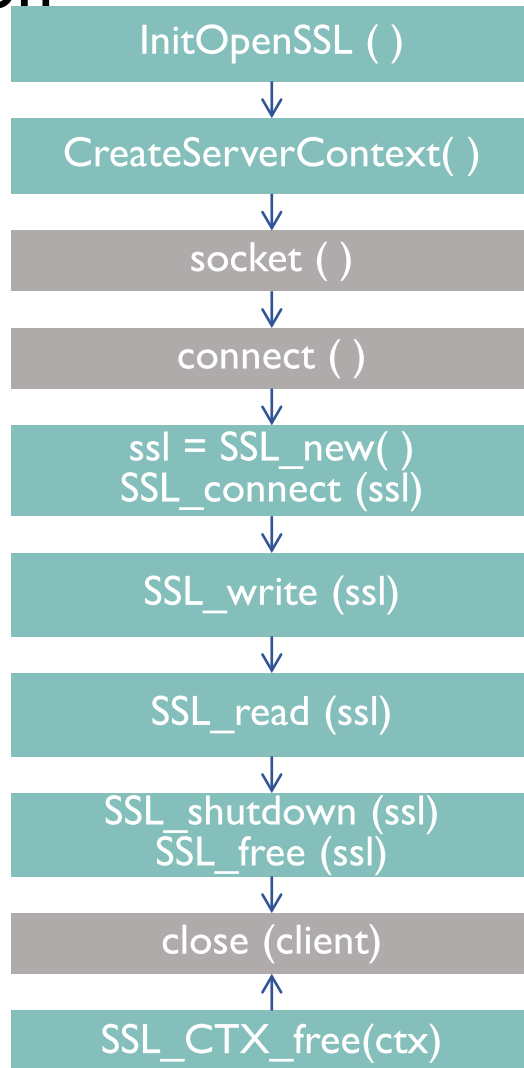


OpenSSL Programming

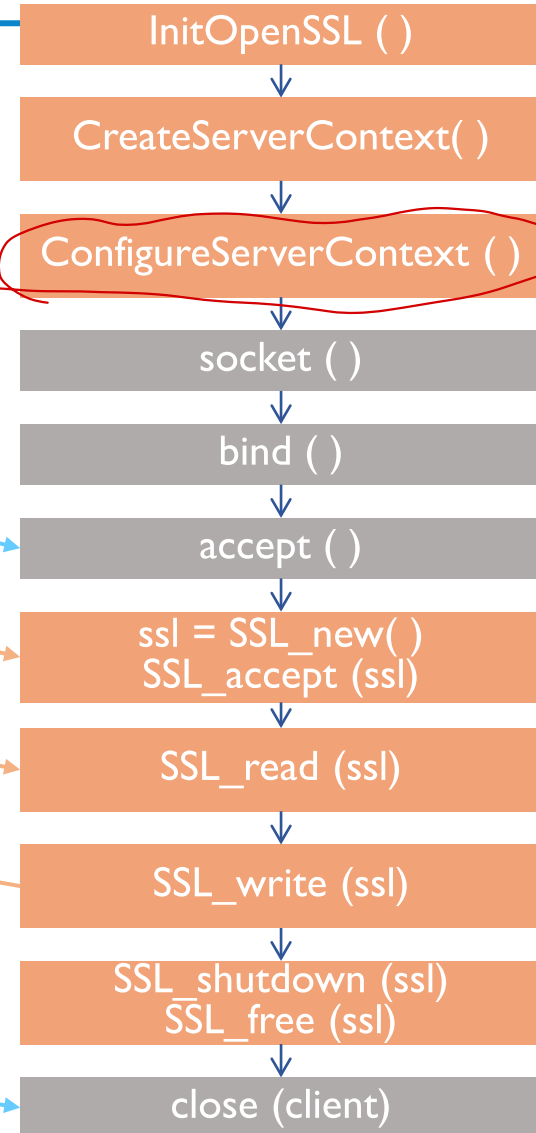
❖ Operation

- initialize the OpenSSL library
- create an SSL context
- Client Socket
- TCP Connection Request
- SSL Connection Request
- echo communicate on SSL conn.
- Shutdown the SSL conn. and Cleanup
- TCP Connection Close

Client



Server



- initialize the OpenSSL library
- create an SSL context
- Set Certificate and Private Key (for server)
- Sockets for the server and new clients
- Create an SSL Connection
- echo communicate on SSL conn.
- Shutdown the SSL conn. and Cleanup
- TCP Connection Close



OpenSSL Programming

❖ Server

- `tcpEchoSSLserver.c`

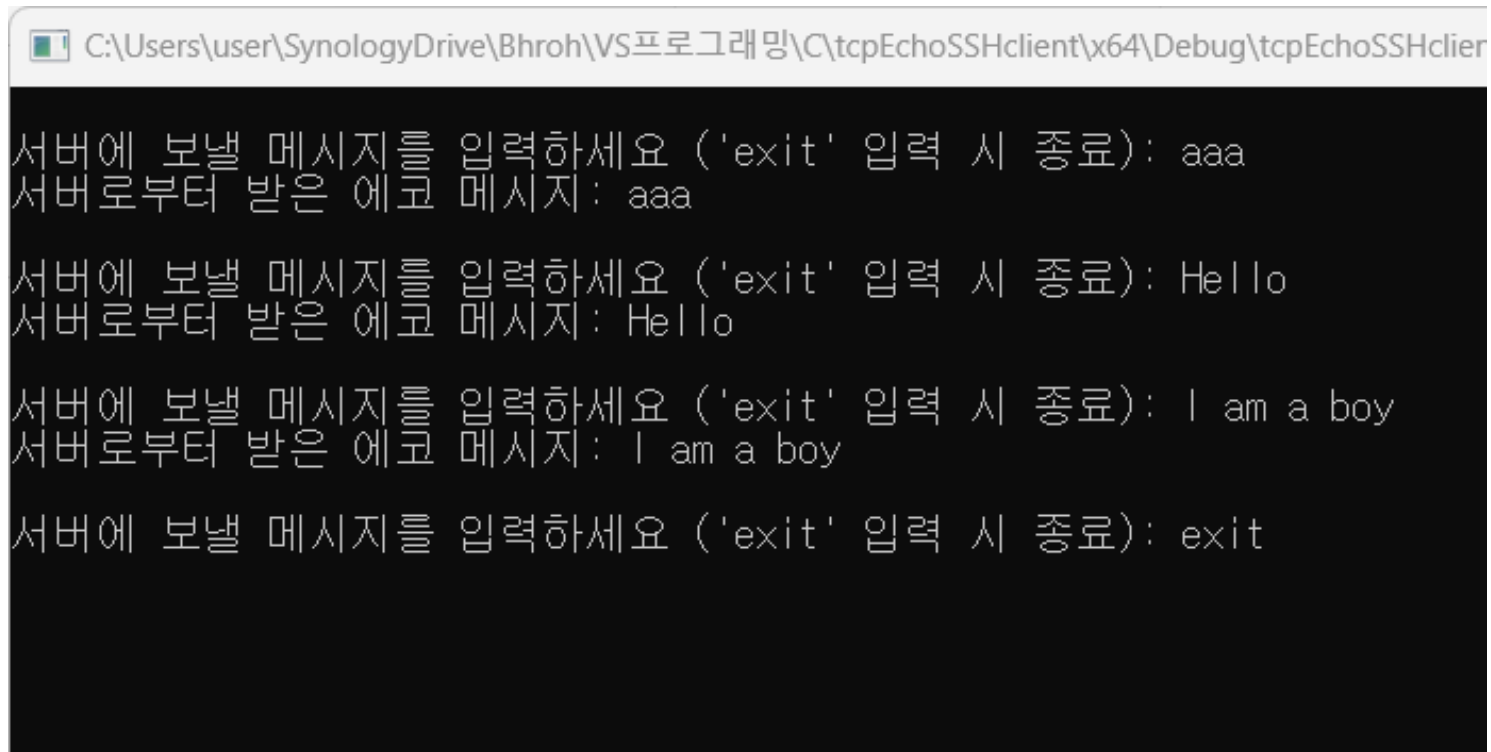
❖ Client

- `tcpEchoSSLclient.c`

OpenSSL Programming

❖ Run (1/3)

- client



```
C:\Users\user\SynologyDrive\Bhroh\VS프로그래밍\C\tcpEchoSSHclient\x64\Debug\tcpEchoSSHclier
서버에 보낼 메시지를 입력하세요 ('exit' 입력 시 종료): aaa
서버로부터 받은 에코 메시지: aaa

서버에 보낼 메시지를 입력하세요 ('exit' 입력 시 종료): Hello
서버로부터 받은 에코 메시지: Hello

서버에 보낼 메시지를 입력하세요 ('exit' 입력 시 종료): I am a boy
서버로부터 받은 에코 메시지: I am a boy

서버에 보낼 메시지를 입력하세요 ('exit' 입력 시 종료): exit
```

OpenSSL Pr

- ❖ Run (2/3)
 - Wireshark Capture

The image shows a Wireshark network capture window titled "Capturing from Wi-Fi". The filter bar at the top shows "tcp.port==9999". The main packet list displays 27 packets. A red box highlights a sequence of packets (17094-17097) showing a TLS handshake and a connection reset. The packet details pane for packet 17094 shows the structure of the TLSv1.3 record, with a red box highlighting the "Application Data" field.

No.	Time	Source	Destination	Protocol	Length	Info
7237	94.152233	192.168.0.12	192.168.0.10	TCP	66	58797 → 9999 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
7241	94.155486	192.168.0.10	192.168.0.12	TCP	66	9999 → 58797 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 S
7245	94.155600	192.168.0.12	192.168.0.10	TCP	54	58797 → 9999 [ACK] Seq=1 Ack=1 Win=131328 Len=0
7254	94.156101	192.168.0.12	192.168.0.10	TLSv1.3	353	Client Hello
7327	94.187582	192.168.0.10	192.168.0.12	TLSv1.3	1514	Server Hello, Change Cipher Spec, Application Data
7329	94.187582	192.168.0.10	192.168.0.12	TLSv1.3	797	Application Data, Application Data, Application Data
7331	94.187683	192.168.0.12	192.168.0.10	TCP	54	58797 → 9999 [ACK] Seq=300 Ack=2204 Win=131328 Len=0
7346	94.189503	192.168.0.12	192.168.0.10	TLSv1.3	134	Change Cipher Spec, Application Data
7362	94.195032	192.168.0.10	192.168.0.12	TLSv1.3	309	Application Data
7492	94.236400	192.168.0.12	192.168.0.10	TCP	54	58797 → 9999 [ACK] Seq=380 Ack=2459 Win=131072 Len=0
7494	94.238317	192.168.0.10	192.168.0.12	TLSv1.3	309	Application Data
7515	94.281998	192.168.0.12	192.168.0.10	TCP	54	58797 → 9999 [ACK] Seq=380 Ack=2714 Win=130816 Len=0
9905	100.691679	192.168.0.12	192.168.0.10	TLSv1.3	79	Application Data
9906	100.693471	192.168.0.10	192.168.0.12	TLSv1.3	79	Application Data
9910	100.746707	192.168.0.12	192.168.0.10	TCP	54	58797 → 9999 [ACK] Seq=405 Ack=2739 Win=130816 Len=0
16931	122.805518	192.168.0.12	192.168.0.10	TLSv1.3	81	Application Data
16933	122.809635	192.168.0.10	192.168.0.12	TLSv1.3	81	Application Data
16934	122.853649	192.168.0.12	192.168.0.10	TCP	54	58797 → 9999 [ACK] Seq=432 Ack=2766 Win=130816 Len=0
17094	127.499278	192.168.0.12	192.168.0.10	TLSv1.3	86	Application Data
17096	127.509438	192.168.0.10	192.168.0.12	TLSv1.3	86	Application Data
17098	127.557731	192.168.0.12	192.168.0.10	TCP	54	58797 → 9999 [ACK] Seq=464 Ack=2798 Win=130560 Len=0
21091	153.287351	192.168.0.12	192.168.0.10	TLSv1.3	78	Application Data
21092	153.287542	192.168.0.12	192.168.0.10	TCP	54	58797 → 9999 [FIN, ACK] Seq=488 Ack=2798 Win=130560 Len=0
21093	153.292307	192.168.0.10	192.168.0.12	TLSv1.3	78	Application Data
21094	153.292307	192.168.0.10	192.168.0.12	TCP	54	9999 → 58797 [FIN, ACK] Seq=2822 Ack=488 Win=1049088 Len=0
21095	153.292351	192.168.0.12	192.168.0.10	TCP	54	58797 → 9999 [RST, ACK] Seq=489 Ack=2822 Win=0 Len=0
21097	153.295212	192.168.0.10	192.168.0.12	TCP	54	9999 → 58797 [ACK] Seq=2823 Ack=489 Win=1049088 Len=0

Frame 17094: 86 bytes on wire (688 bits), 86 bytes captured (688 bits) on interface 0
Ethernet II, Src: Intel_bb:97:a7 (70:cf:49:bb:97:a7), Dst: Intel_bb:97:a7 (70:cf:49:bb:97:a7)
Internet Protocol Version 4, Src: 192.168.0.12, Dst: 192.168.0.10
Transmission Control Protocol, Src Port: 58797, Dst Port: 9999
Transport Layer Security

0000 14 18 c3 7e dc 0e 70 cf 49 bb 97 a7 08 00 45 00p. I.....E.
0010 00 48 83 ab 40 00 80 06 00 00 c0 a8 00 0c c0 a8 ..H..@.....
0020 00 0a e5 ad 27 0f 2e fe 0f 81 0a 0c 87 9f 50 18p.
0030 01 ff 81 a1 00 00 17 03 03 00 1b cc 73 f0 13 43S..C
0040 22 bd b1 6a f5 d6 78 5b 75 30 d6 53 8c a7 2e af "...j...x[u0.S...
0050 89 3d 3b 1f b5 77 ..=;...w

OpenSSL Programming

❖ Run (3/3)

- Wireshark Capture
 - without SSL

The image shows a Wireshark network traffic capture window. The top menu bar includes File, Edit, View, Go, Capture, Analyze, Statistics, Telephony, Wireless, Tools, and Help. The status bar at the top indicates 'Capturing from Wi-Fi'. The main display area shows a list of captured packets. The 'tcp.port == 9999' filter is applied. The packet list shows a series of TCP packets between 192.168.0.12 and 192.168.0.10. The 'Protocol' column for these packets is highlighted with a red box. The packet details pane shows the selected packet (No. 989) with its structure: Ethernet II, Internet Protocol Version 4, and Transmission Control Protocol. The data field shows the raw data: 4920616d206120626f79. The packet bytes pane shows the raw data in hexadecimal and ASCII. The ASCII representation of the data is 'I am a boy', which is highlighted with a red box. The status bar at the bottom shows 'Data (data.data), 10 bytes', the URL 'http://mmcn.ajou.ac.kr', and other statistics like 'Packets: 21841 · Displayed: 16 (0.1%)' and 'Profile: Default'.

No.	Time	Source	Destination	Protocol	Length	Info
394	5.567073	192.168.0.12	192.168.0.10	TCP	66	58403 → 9999 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
395	5.569356	192.168.0.10	192.168.0.12	TCP	66	9999 → 58403 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 S
396	5.569407	192.168.0.12	192.168.0.10	TCP	54	58403 → 9999 [ACK] Seq=1 Ack=1 Win=131328 Len=0
536	10.482809	192.168.0.12	192.168.0.10	TCP	57	58403 → 9999 [PSH, ACK] Seq=1 Ack=1 Win=131328 Len=3
537	10.486340	192.168.0.10	192.168.0.12	TCP	57	9999 → 58403 [PSH, ACK] Seq=1 Ack=4 Win=1049600 Len=3
542	10.533680	192.168.0.12	192.168.0.10	TCP	54	58403 → 9999 [ACK] Seq=4 Ack=4 Win=131328 Len=0
660	13.745804	192.168.0.12	192.168.0.10	TCP	59	58403 → 9999 [PSH, ACK] Seq=4 Ack=4 Win=131328 Len=5
662	13.748788	192.168.0.10	192.168.0.12	TCP	59	9999 → 58403 [PSH, ACK] Seq=4 Ack=9 Win=1049600 Len=5
673	13.800620	192.168.0.12	192.168.0.10	TCP	54	58403 → 9999 [ACK] Seq=9 Ack=9 Win=131328 Len=0
987	21.330444	192.168.0.12	192.168.0.10	TCP	64	58403 → 9999 [PSH, ACK] Seq=9 Ack=9 Win=131328 Len=10
989	21.336352	192.168.0.10	192.168.0.12	TCP	64	9999 → 58403 [PSH, ACK] Seq=9 Ack=19 Win=1049600 Len=10
994	21.382026	192.168.0.12	192.168.0.10	TCP	54	58403 → 9999 [ACK] Seq=19 Ack=19 Win=131328 Len=0
1089	23.842684	192.168.0.12	192.168.0.10	TCP	54	58403 → 9999 [FIN, ACK] Seq=19 Ack=19 Win=131328 Len=0
1090	23.844893	192.168.0.10	192.168.0.12	TCP	54	9999 → 58403 [ACK] Seq=19 Ack=20 Win=1049600 Len=0
1091	23.845015	192.168.0.10	192.168.0.12	TCP	54	9999 → 58403 [FIN, ACK] Seq=19 Ack=20 Win=1049600 Len=0
1092	23.845040	192.168.0.12	192.168.0.10	TCP	54	58403 → 9999 [ACK] Seq=20 Ack=20 Win=131328 Len=0

Frame 989: 64 bytes on wire (512 bits), 64 bytes captured (512 bits) on interface 0
Ethernet II, Src: Intel_7e:dc:0e (14:18:c3:7e:dc:0e), Dst: Intel_7e:dc:0e (14:18:c3:7e:dc:0e)
Internet Protocol Version 4, Src: 192.168.0.10, Dst: 192.168.0.12
Transmission Control Protocol, Src Port: 9999, Dst Port: 58403
Data (10 bytes)
Data: 4920616d206120626f79
[Length: 10]

0000 70 cf 49 bb 97 a7 14 18 c3 7e dc 0e 08 00 45 00 p-I.....~....E.
0010 00 32 38 0e 40 00 80 06 41 51 c0 a8 00 0a c0 a8 .28.0....AQ.....
0020 00 0c 27 0f e4 23 30 b3 f1 1b 25 cc 37 d3 50 18 ...'.#0...%.7.P.
0030 10 04 38 ec 00 00 49 20 61 6d 20 61 20 62 6f 79 ..8...I am a boy

Questions ?

MR-IoT/AI Convergence

Future Internet & 5G/6G

Intelligent Networking with AI

MMcN



Mobile **Multimedia Convergence** Network Lab.

Homepage: <http://mmcn.ajou.ac.kr>

facebook: <http://www.facebook.com/#!/groups/190702010947314>

